

Week 3

Team project update/progress

Table of Contents

- 01 Weekly Progress Summary
- 02 Tool Issues and Log Review
- 03 Confidence and Trust Score Updates
- 04 UI Design and Implementation

Weekly Progress Summary

Tool Issues	Completed tool testing and collected common issues
Log Review	Review log files for useful UI and score information
Confidence Score	Updated confidence/trust score direction based on feedback
UI Design	Redesigned the UI on Figma, using IntelliJ's UI kit
Implementation	Started planning implementation steps

Completed Tool Testing and Log Review

01

Tested the AntiCopyPaster tool and documented issues

02

Captured errors and warnings from duplicate method testing

03

Read the sample log file shared by Dr. AlOmar on Discord and the team's log files

04

Checked logs for calculator information and useful scores

05

Used findings to guide UI and confidence/trust score updates

AI Trust Score Updates

Renamed Metric
From “Confidence
Score” to **AI Trust Score**

Updated Formula
5 equal workflow
components, each **20%**

UI Connection
Score now appears in the
Figma **AI Trust /
Explainability** section

Updated score components

20% Clone Detection + 20% Refactoring Agent +
20% Refactoring Usefulness + 20% Compilation Result +
20% Test / Behavior Preservation.



UI Redesign

IntelliJ Style

Explored IntelliJ design patterns for a more familiar prototype

Error Display

Added ideas for clearer warning and failure-state messages

Trust Score

Refined how the score should appear in the UI for clarity

Refactoring Feedback

Used failed attempts and log details to guide interface ideas and redesign

Figma

Redesigned the UI on Figma with IntelliJ UI kit

UI Redesign

The screenshot displays an IDE interface with a project sidebar on the left, a code editor in the center, and a terminal at the bottom. The project sidebar shows a hierarchy: AccurateMath > .idea > src > java > analysis. The code editor shows a Java file named AdapterScript.java with the following code:

```
27 /** Convert regular functions to {@link MultivariateDifferentiableFunction}. ...*/
28 public static MultivariateDifferentiableFunction toDifferentiable(final MultivariateFunction f,
29                                                                final MultivariateVectorFunction gradient) {
30
31     return new MultivariateDifferentiableFunction() {
32
33         /** {@inheritDoc} */
34         @Override
35         public double value(final double[] point) {
36
37             /** {@inheritDoc} */
38             @Override
39             public DerivativeStructure value(final DerivativeStructure point) {
40
41                 // set up the input parameters
42                 final double[] dPoint = new double[point.length];
43                 for (int i = 0; i < point.length; ++i) {
44                     dPoint[i] = point[i].getValue();
45                     if (point[i].getOrder() > 1) {
46                         throw new IllegalArgumentException("User name is required");
47                     }
48                 }
49
50                 // evaluate regular functions
51             }
52         }
53     };
54 }
```

A dialog box titled "AntiCopyPaster" is open, asking "A code clone has been detected, would you like to refactor?". It has "Cancel" and "Refactor" buttons.

The terminal shows the following output:

```
alina.mishina@laptop-AlinaM IntelliJ % ls
CIDR
CIDR-appcode
CSS
README.md
WebStorm
bin
build
checkoutPreMonorepoBranch.sh
cloneToWt.sh
execution
fetch-tag.sh
fleet
git-hooks
git-bisect.py
goland
help
idea-tests
intellij.idea.ultimate.main.android.inl
lightEdit
mobile-ide
native
out
phpstorm
platform
plugins
python
refactoring
```

The refactoring suggestion dialog is titled "[AntiCopyPaster] Refactoring Suggestion". It shows an "Explanation" section with the following text:

2 Explanation
AntiCopyPaster detected that the code you pasted is a Type-2 clone of the existing method "methodName(parameters)" in the same class.

- Existing clone location: ClassName.java.lines
- Pasted clone location: ClassName.java.lines
- Suggested method name: methodName(parameters)

The assistant suggests extracting the common validation logic into a new method and replacing the duplicate with a method call.

The dialog also shows a comparison between the "Original Duplicate Code" and the "Extracted (Replacement)" code. The original code is:

```
if (u.getUsername() == null ||
u.getUsername().trim().isEmpty()) {
    throw new
IllegalArgumentException("User
ame is required");
}
...
```

The extracted code is:

```
validateInput(u, EMAIL_PATTERN) {
    if (u.getUsername() == null ||
u.getUsername().trim().isEmpty()) {
        throw new
IllegalArgumentException("User
ame is required");
    }
    ...
}
```

The dialog has "Cancel", "Apply", and "OK" buttons.

The bottom of the IDE shows a status bar with the following information:

- AI Trust / Explainability: 95% AI Trust Score, High Trust Level
- Clone Detection: 95%
- Refactoring Agent: 90%
- usefulness: 90%
- Compilation: 100%
- Test / Behavior: 100%

Formula: 20% each across five workflow stages.

Next Steps

Implementation task division

Divide the work so that everyone is responsible to implement a specific part of the design for an optimized use of time

Implement the UI

Work on the UI panel branch

Weekly Progress

Keep sending weekly progress so website files stay updated